

Descubrimiento de ecuaciones con HPC para Gemelos Digitales

Un enfoque híbrido de IA aceleradora y Computación de Alto Desempeño

José Sebastian Sasías, MSc. Ing.

jose.sasias@fing.edu.uy

Doctorado - PEDECIBA Informática

Facultad de Ingeniería - Universidad de la República, Uruguay

Agosto, 2026

Resumen

El descubrimiento automático de ecuaciones diferenciales a partir de datos es un pilar fundamental en la construcción de gemelos digitales (GD) transparentes y fiables. Este trabajo presenta el desarrollo de un sistema híbrido que combina el conocimiento físico del dominio (familias de modelos) con técnicas de optimización numérica y computación de altas prestaciones (HPC) para descubrir ecuaciones interpretables que representen la dinámica de sistemas mecánicos y electro-termo-mecánicos.

El proyecto se inició explorando técnicas de inteligencia artificial (redes neuronales de grafos, autoencoders variacionales y aprendizaje por refuerzo) como aceleradores del proceso de descubrimiento. Si bien estas técnicas mostraron resultados prometedores en sistemas estructurados, su efectividad se degradó ante sistemas con alta sensibilidad a parámetros o ruido, lo que llevó a un cambio de enfoque hacia la optimización paramétrica paralela basada en familias de modelos (matemáticas y físicas) que fueron identificados en el proceso asistido con IA.

Se implementó un descubridor en C con MPI que utiliza 10 familias de modelos (5 físicas y 5 matemáticas) y un esquema maestro-trabajador con balance de carga dinámico. El sistema fue validado en sistemas de referencia (masa-resorte, Duffing, Lorenz, motor DC) y en un caso de estudio realista de vibraciones de aerogenerador, logrando errores de ajuste del orden de 10^{-7} a 10^{-12} .

Los experimentos de escalabilidad mostraron un speedup máximo de 5.38x con 8 procesos, con una eficiencia del 67% en ese punto. La curva de Amdahl ajustada indica una fracción paralelizable $fp = 0,922$, con un speedup máximo teórico de 12.82x, aunque en la práctica el overhead de comunicación y la contención de recursos limitan el speedup a 5.38x. Se identificaron limitaciones en sistemas multiescala (rigidez temporal) y se propone un enfoque modular como extensión futura.

El trabajo demuestra la viabilidad de combinar el conocimiento físico con la optimización paramétrica paralela para el descubrimiento de ecuaciones, y sienta las bases para la construcción de gemelos digitales basados en modelos interpretables. Los códigos fuente, scripts de generación de datos y herramientas de visualización están disponibles en el repositorio del proyecto [\[12\]](#).

Índice general

1. Antecedentes y trayectoria	3
1.1. Origen y motivación	3
1.2. Fase 1: exploración de IA y regresión simbólica	3
1.2.1. Uso de redes neuronales de grafos (GNN)	3
1.2.2. Autoencoders variacionales (VAE) y aprendizaje por refuerzo (RL)	4
1.2.3. Regresión simbólica con familias matemáticas	4
1.3. Fase 2: Descubridor con familias físicas	4
1.4. Fase 3: Paralelización con MPI	4
1.5. Fase 4: Limitaciones y propuesta modular	5
2. Marco teórico y estado de la técnica	6
2.1. Gemelos digitales: componentes y desafíos	6
2.2. Técnicas de descubrimiento de ecuaciones	6
2.2.1. SINDy (Sparse Identification of Nonlinear Dynamics)	6
2.2.2. Regresión simbólica	6
2.2.3. Optimización paramétrica con familias de modelos	6
2.3. Computación de altas prestaciones (HPC)	7
2.3.1. Métricas de rendimiento	7
2.4. Sistemas multiescala y rigidez temporal	7
3. Metodología y diseño del sistema	8
3.1. Formulación matemática del problema	8
3.2. Arquitectura general del descubridor	9
3.2.1. Flujo de ejecución	9
3.3. Implementación de familias	9
3.3.1. Jerarquía de clases	9
3.3.2. Familias físicas	10
3.3.3. Familias matemáticas	10
3.4. Optimización de parámetros	10
3.5. Paralelización con MPI	10
3.6. Generación de salidas	10
4. Experimentos y resultados	12
4.1. Sistemas de referencia	12
4.2. Caso de estudio: vibraciones de aerogenerador	12
4.2.1. Gráficos de comparación	12
4.3. Experimentos de speedup	14
4.3.1. Definición de Speedup y Eficiencia	14
4.3.2. Resultados de speedup	14

4.3.3.	Curva de Amdahl y fracción paralelizable	14
4.3.4.	Casos procesados	15
4.3.5.	Análisis de escalabilidad	18
5.	Discusión y lecciones aprendidas	20
5.1.	Interpretación de los resultados de speedup	20
5.2.	Comparación con otros enfoques	20
5.2.1.	SINDy	20
5.2.2.	Regresión simbólica	20
5.2.3.	PINNs	20
5.3.	Aprendizaje del proceso	21
6.	Conclusiones y trabajo futuro	22
6.1.	Resumen de logros	22
6.2.	Contribuciones del trabajo	22
6.3.	Trabajo futuro	23
A.	Código: descubridor MPI en C	24
A.1.	Estructuras de datos para comunicación MPI	24
A.2.	Función de procesamiento de tarea (worker)	25
B.	Scripts de generación de gráficos	26
B.1.	Script Python para gráficos de curvas y errores	26
B.2.	Script Python para medición de speedup	27
C.	Resultados adicionales	28
C.1.	Ejemplo de ecuación para falla de holgura	28
C.2.	Comparativas secuenciales entre Python y C	28
D.	Glosario	29

Capítulo 1

Antecedentes y trayectoria

1.1. Origen y motivación

El proyecto nace de la necesidad de construir gemelos digitales (GD) de sistemas físicos complejos, como aerogeneradores, motores eléctricos o sistemas robóticos, mediante modelos matemáticos que sean precisos, interpretables y computacionalmente eficientes. Tradicionalmente, la obtención de estos modelos requiere un costoso trabajo manual de identificación, apoyado en el conocimiento experto de la física subyacente. El descubrimiento automático de ecuaciones diferenciales a partir de datos de entrada/salida ofrece una alternativa prometedora, pero enfrenta desafíos en términos de escalabilidad, robustez al ruido y capacidad para manejar sistemas con dinámicas muy diversas.

El presente proyecto se inició con la hipótesis de que las técnicas de inteligencia artificial, especialmente las redes neuronales de grafos (GNN), los autoencoders variacionales (VAE) y el aprendizaje por refuerzo (RL), podían acelerar significativamente el proceso de descubrimiento, filtrando modelos poco prometedores y guiando la búsqueda. Sin embargo, a lo largo de la investigación, se encontraron limitaciones importantes que llevaron a un cambio de enfoque hacia la optimización paramétrica paralela basada en familias de modelos (físicas y matemáticas) y el uso intensivo de HPC.

Si bien la solución finalmente programada no utilizó estos conceptos de IA como parte de su implementación, este proceso permitió identificar las familias de modelos más relevantes que pueden conformar una base de representación de muchos sistemas. Dichas familias se conservaron y usaron para la solución final con HPC.

1.2. Fase 1: exploración de IA y regresión simbólica

1.2.1. Uso de redes neuronales de grafos (GNN)

Inicialmente, se exploró el uso de GNN para predecir el error de ajuste de modelos candidatos sin necesidad de simularlos. Se implementó un GNN que toma como entrada la representación en grafo de la ecuación (árbol de expresión) y devuelve una estimación del error. Los experimentos se realizaron con el sistema masa-resorte-amortiguador, generando miles de modelos aleatorios. Los resultados mostraron que el GNN lograba una correlación de Pearson de 0.85 entre la predicción y el error real, lo que permitía filtrar alrededor del 70 % de los modelos sin perder los mejores. Sin embargo, cuando se aplicaba a sistemas

más complejos (Duffing con ruido), la correlación caía a 0.6, y el filtrado era ineficaz. Además, el entrenamiento del GNN requería una gran cantidad de evaluaciones costosas, lo que contrarrestaba el ahorro en la fase de búsqueda.

1.2.2. Autoencoders variacionales (VAE) y aprendizaje por refuerzo (RL)

Se exploraron también VAE para generar mutaciones inteligentes de modelos y RL para ajustar dinámicamente los hiperparámetros del algoritmo de búsqueda. El VAE generaba modelos que, aunque diversos, tendían a sobreajustar y no mejoraban significativamente la convergencia respecto a mutaciones aleatorias. El agente RL (PPO) fue capaz de ajustar la tasa de mutación y la presión selectiva, pero el beneficio obtenido fue marginal (aprox. 10 % de mejora) y el coste de entrenamiento era elevado.

1.2.3. Regresión simbólica con familias matemáticas

Ante las limitaciones de los enfoques puramente basados en IA, se optó por un método más clásico: la regresión simbólica con un conjunto de familias matemáticas predefinidas. Se implementaron familias polinómicas, racionales, Fourier y exponenciales. Estas familias se optimizaron mediante evolución diferencial y búsqueda local. Los resultados obtenidos por este camino fueron alentadores:

- En el sistema masa-resorte-amortiguador, la familia polinómica de grado 2 recuperó o descubrió exactamente la ecuación $\ddot{x} = -2,0x - 0,5\dot{x} + F(t)$ con un error de 10^{-15} .
- En el oscilador de Duffing, la familia polinómica de grado 3 encontró la ecuación: $\ddot{x} = 1,0x - 0,2\dot{x} - 1,0x^3 + F(t)$.
- En el sistema de Lorenz, las familias racionales y polinómicas de grado 2 recuperaron las tres ecuaciones con errores similares.

1.3. Fase 2: Descubridor con familias físicas

Aunque las familias matemáticas funcionaban bien, los modelos resultantes a menudo carecían de interpretabilidad física y podían ser inestables fuera del rango de entrenamiento. Para sistemas donde se conoce la física subyacente (como vibraciones de aerogeneradores), se implementaron cinco familias físicas: OsciladorSimple, Duffing, AmortiguamientoNoLineal, ForzamientoParametrico e Impacto. Cada familia se define como una clase con métodos para evaluar el error y simular la respuesta.

Las familias físicas se probaron en los mismos sistemas de referencia y en datos sintéticos de aerogenerador, logrando errores de ajuste del orden de 10^{-7} a 10^{-12} .

1.4. Fase 3: Paralelización con MPI

Dado que la evaluación de cada familia sobre cada archivo es independiente, se implementó un esquema maestro-trabajador con MPI. El maestro distribuye las tareas (familia + archivo) entre los trabajadores, estos ejecutan la optimización y devuelven los resultados. El maestro recolecta y selecciona el mejor modelo. Este esquema permite balance de carga dinámico y tolerancia a fallos.

1.5. Fase 4: Limitaciones y propuesta modular

Al intentar aplicar el descubridor al motor DC con efectos térmicos y fricción de Coulomb, se encontró que las familias físicas implementadas no podían capturar la dinámica térmica porque esta ocurre en una escala de tiempo mucho más lenta que la eléctrica.

Además, en situaciones como las de un brazo robótico de 2 GDL y el coeficiente de potencia $C_p(\lambda, \beta)$ del aerogenerador presentan estructuras que no pueden ser representadas por una única ecuación escalar. Para superar estas limitaciones, se propone un enfoque modular que divide el sistema en subsistemas con escalas temporales similares, los descubre por separado y luego los acopla.

El referido acople entre módulos y representaciones descubiertas es a través de un modelo de transferencia de energía. Como el enfoque del trabajo es descubrir expresiones de las ecuaciones diferenciales que rigen comportamientos de máquinas que se desean implementar como gemelos digitales. Incluso la propia vinculación por balance energético es otro problema del mismo tipo, cuyas expresiones pueden ser descubiertas mediante la misma técnica de paralelización y uso de familias de representación. Por tales motivos, la actividad y alcances se enfocarán en descubrimientos modulares.

Capítulo 2

Marco teórico y estado de la técnica

2.1. Gemelos digitales: componentes y desafíos

Un gemelo digital (GD) es una representación virtual de un sistema físico que se actualiza dinámicamente con datos del mundo real, permitiendo simulación, predicción, control y diagnóstico. Los GD se componen de: el sistema físico (activos, sensores, actuadores), el modelo virtual (ecuaciones, simulaciones) y la conexión (flujo de datos en tiempo real). El desafío principal es la obtención de modelos matemáticos precisos e interpretables que modelen debidamente al dispositivo físico y sean computacionalmente implementables.

2.2. Técnicas de descubrimiento de ecuaciones

2.2.1. SINDy (Sparse Identification of Nonlinear Dynamics)

SINDy [1] es un método que descubre ecuaciones diferenciales a partir de datos mediante la selección esparsa de términos en una biblioteca de funciones candidatas. Funciona bien para sistemas que pueden expresarse como combinación lineal de términos de la biblioteca, pero falla cuando la estructura no está en la biblioteca o cuando hay múltiples escalas temporales.

2.2.2. Regresión simbólica

La regresión simbólica [2] busca expresiones matemáticas explícitas que mejor se ajusten a los datos, utilizando algoritmos genéticos o búsqueda en árbol. Es flexible pero computacionalmente costosa y puede producir modelos no interpretables.

2.2.3. Optimización paramétrica con familias de modelos

El enfoque adoptado en este proyecto consiste en restringir el espacio de búsqueda a un conjunto de familias de modelos con estructura conocida (físicas o matemáticas) y optimizar sus parámetros. Este enfoque combina la interpretabilidad de los modelos físicos con la flexibilidad de las familias matemáticas, y es fácilmente paralelizable.

2.3. Computación de altas prestaciones (HPC)

El problema de descubrimiento de ecuaciones es embarazosamente paralelo: cada evaluación de una familia sobre un archivo es independiente. Esto permite distribuir las tareas entre múltiples procesos sin necesidad de comunicación frecuente. Se utiliza MPI para implementar un esquema maestro-trabajador.

2.3.1. Métricas de rendimiento

- **Speedup:** $S_N = T_1/T_N$, donde T_1 es el tiempo secuencial y T_N el tiempo con N procesos.
- **Eficiencia:** $E_N = S_N/N \times 100\%$.
- **Ley de Amdahl:** $S(p) = 1/((1 - f_p) + f_p/p)$, donde f_p es la fracción paralelizable.

2.4. Sistemas multiescala y rigidez temporal

Los sistemas multiescala presentan constantes de tiempo muy diferentes (ej. eléctrica: ms, mecánica: s, térmica: min). Esto dificulta el descubrimiento de ecuaciones porque la dinámica lenta es indistinguible del ruido numérico de las dinámicas rápidas. El enfoque modular propuesto aborda este problema dividiendo el sistema en subsistemas con escalas de tiempo similares.

Capítulo 3

Metodología y diseño del sistema

3.1. Formulación matemática del problema

El problema fundamental que aborda este trabajo es el descubrimiento automático de ecuaciones diferenciales que describan la dinámica de un sistema físico a partir de mediciones de sus variables de entrada y salida.

Se considera un sistema dinámico con:

- Señales de entrada: $\mathbf{u}(t) \in \mathbb{R}^m$
- Señales de salida: $\mathbf{y}(t) \in \mathbb{R}^p$

El objetivo es encontrar una función $\mathcal{F}$ y un conjunto de parámetros $\boldsymbol{\theta}$ tales que se cumpla la siguiente identidad para todo instante de tiempo t :

$$\mathcal{F}[\mathbf{y}(t), \dot{\mathbf{y}}(t), \ddot{\mathbf{y}}(t), \dots, \mathbf{u}(t), \boldsymbol{\theta}] = 0 \quad \forall t.$$

Esta formulación es genérica y abarca una amplia clase de sistemas dinámicos. En este trabajo, el interés se centra en sistemas mecatrónicos y electro-termo-mecánicos, los cuales no son invariantes en el tiempo y presentan una vinculación operativa entre las señales de salida y sus derivadas temporales.

En la práctica, se restringe el estudio a sistemas que pueden representarse mediante ecuaciones diferenciales ordinarias (EDO) de primer orden en forma de espacio de estados:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t), \boldsymbol{\theta}),$$

donde $\mathbf{x}(t) \in \mathbb{R}^n$ es el vector de estado del sistema. El problema de descubrimiento consiste entonces en determinar tanto la estructura de la función $\mathbf{f}$ como los valores de los parámetros $\boldsymbol{\theta}$ que minimicen el error entre la simulación del modelo y los datos observados.

Para hacer el problema abordable computacionalmente, se restringe el espacio de búsqueda a un conjunto de *familias de modelos* con estructura conocida (físicas o matemáticas), y se optimizan sus parámetros mediante algoritmos numéricos. Esta estrategia permite explorar el espacio de soluciones de manera eficiente y paralelizable, como se describe en las secciones siguientes.

3.2. Arquitectura general del descubridor

El sistema de descubrimiento de ecuaciones se organiza en los siguientes módulos:

1. **Módulo de entrada:** Lee los datos de entrada (archivos CSV con series temporales de vibración y fuerza), valida su formato y los prepara para la optimización.
2. **Módulo de familias:** Contiene la implementación de todas las familias de modelos (físicas y matemáticas), cada una con su propia función de evaluación y simulación.
3. **Módulo de optimización:** Para cada familia y archivo, optimiza los parámetros mediante un algoritmo de dos fases (evolución diferencial + búsqueda local).
4. **Módulo de selección:** Una vez obtenidos todos los resultados, selecciona el mejor modelo según criterios predefinidos (error, AIC, estabilidad).
5. **Módulo de paralelización:** Distribuye las tareas entre los procesos MPI y coordina la recolección de resultados.
6. **Módulo de salida:** Genera logs, gráficos de comparación y reportes en formato texto y CSV.

3.2.1. Flujo de ejecución

El flujo principal del programa (ejecutado por el proceso maestro) es:

1. Leer la lista de archivos de datos del directorio de entrada.
2. Para cada archivo y cada familia:
 - a) Si se ejecuta en modo secuencial (1 proceso): optimizar localmente.
 - b) Si se ejecuta en modo paralelo: asignar la tarea a un worker.
3. Recolectar los resultados de todos los workers.
4. Para cada archivo, seleccionar la familia con menor error.
5. Generar gráficos de comparación para el mejor modelo de cada archivo.
6. Escribir logs y reportes.

3.3. Implementación de familias

3.3.1. Jerarquía de clases

Todas las familias heredan de la clase base `FamiliaBase`, que define la interfaz común: métodos `evaluar`, `simular` y `generar_ecuacion`. Cada familia concreta implementa los métodos utilizando `scipy.integrate.odeint` (en la versión Python) o un solver RK4 propio [7] (Runge-Kutta de cuarto orden) en la versión C.

3.3.2. Familias físicas

Se han implementado cinco familias físicas que cubren comportamientos típicos de sistemas mecánicos con no linealidades. La Tabla 3.1 resume sus ecuaciones y parámetros.

Cuadro 3.1: Familias físicas implementadas

Familia	Ecuación	Parámetros
Oscilador Simple	$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x = F(t)$	ω_n, ζ
Oscilador Duffing [9]	$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x + \beta x^3 = F(t)$	ω_n, ζ, β
Amortiguamiento No Lineal [10]	$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x + \gamma\dot{x}^3 = F(t)$	ω_n, ζ, γ
Forzamiento Paramétrico	$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2(1 + \epsilon \sin(\omega_p t))x = F(t)$	$\omega_n, \zeta, \epsilon, \omega_p$
Impacto	$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x + F_{\text{imp}}(x) = F(t)$	$\omega_n, \zeta, \text{gap}, k_{\text{imp}}$

3.3.3. Familias matemáticas

Las familias matemáticas se utilizan para la exploración sin hipótesis física. Se implementaron polinómicas (grados 2 y 3), Fourier (2 y 3 armónicos) y exponencial. Estas familias no requieren integración numérica, sino que calculan derivadas numéricas de los datos y ajustan una combinación lineal de funciones.

3.4. Optimización de parámetros

La optimización se realiza en dos fases:

1. **Exploración global:** mediante evolución diferencial (`scipy.optimize.differential_evolution` en Python, implementación propia en C), que permite explorar amplias regiones del espacio de parámetros sin quedar atrapado en mínimos locales.
2. **Refinamiento local:** utilizando el método de Nelder-Mead [8], a partir de la mejor solución encontrada por evolución diferencial.

3.5. Paralelización con MPI

Se implementó un esquema maestro-trabajador dinámico. El maestro distribuye tareas (archivo, familia) a los workers, estos optimizan los parámetros y devuelven los resultados. El balance de carga se logra asignando nuevas tareas a los workers que terminan antes. Se incluye tolerancia a fallos mediante timeout y reintentos.

3.6. Generación de salidas

El sistema genera los siguientes archivos de salida por cada ejecución:

- `resultados.txt`: Lista de los mejores modelos por archivo, con su ecuación y error.
- `log_speedup.json`: Archivo JSON con toda la información de la ejecución (tiempos, familias, parámetros, etc.).

- `tiempos.csv`: Tabla con los tiempos de ejecución para cada combinación (archivo, familia, procesos).
- `speedup.csv`: Tabla con los speedups calculados.
- Gráficos de comparación (datos reales vs modelo descubierto) para el mejor modelo de cada archivo.

Todos los resultados se almacenan en una carpeta con timestamp para evitar sobrescritura.

Capítulo 4

Experimentos y resultados

4.1. Sistemas de referencia

El descubridor fue validado en sistemas de referencia: masa-resorte-amortiguador, oscilador de Duffing, sistema de Lorenz y motor DC. En todos los casos, las ecuaciones descubiertas coincidieron con las ecuaciones originales con errores del orden de 10^{-12} a 10^{-15} . Las pruebas iniciales se hicieron sintetizando datos a partir del modelo matemático (el que luego debería ser descubierto), posteriormente se añadió ruido al proceso para valorar el desempeño del descubrimiento ante estas situaciones.

4.2. Caso de estudio: vibraciones de aerogenerador

A partir de modelos conocidos y/o datos abiertos [11, 13, 14], con el propósito de contar con un conjunto de datos representativos, se generaron datos sintéticos de vibraciones de aerogenerador con cinco tipos de falla: saludable, desgaste de rodamiento, desbalanceo, falla de engranaje y holgura. Estos datos, tenían además un nivel realista de ruido añadido.

Para cada tipo, el descubridor seleccionó la familia más adecuada. La Tabla 4.1 resume los resultados.

Cuadro 4.1: Familias seleccionadas por tipo de falla

Tipo de falla	Familia seleccionada	Error MSE
Saludable	Oscilador Simple	$1,2 \times 10^{-10}$
Desgaste de rodamiento	Duffing	$8,5 \times 10^{-11}$
Desbalanceo	Oscilador Simple	$2,3 \times 10^{-10}$
Falla de engranaje	Forzamiento Paramétrico	$1,1 \times 10^{-9}$
Holgura	Impacto	$3,7 \times 10^{-11}$

4.2.1. Gráficos de comparación

Para cada archivo, el sistema genera un gráfico combinado con dos subplots: arriba la comparación entre los datos reales y el modelo descubierto, y abajo el error puntual $e(t) = \text{real}(t) - \text{modelo}(t)$. La Figura 4.1 muestra un ejemplo representativo para el caso de falla de holgura.

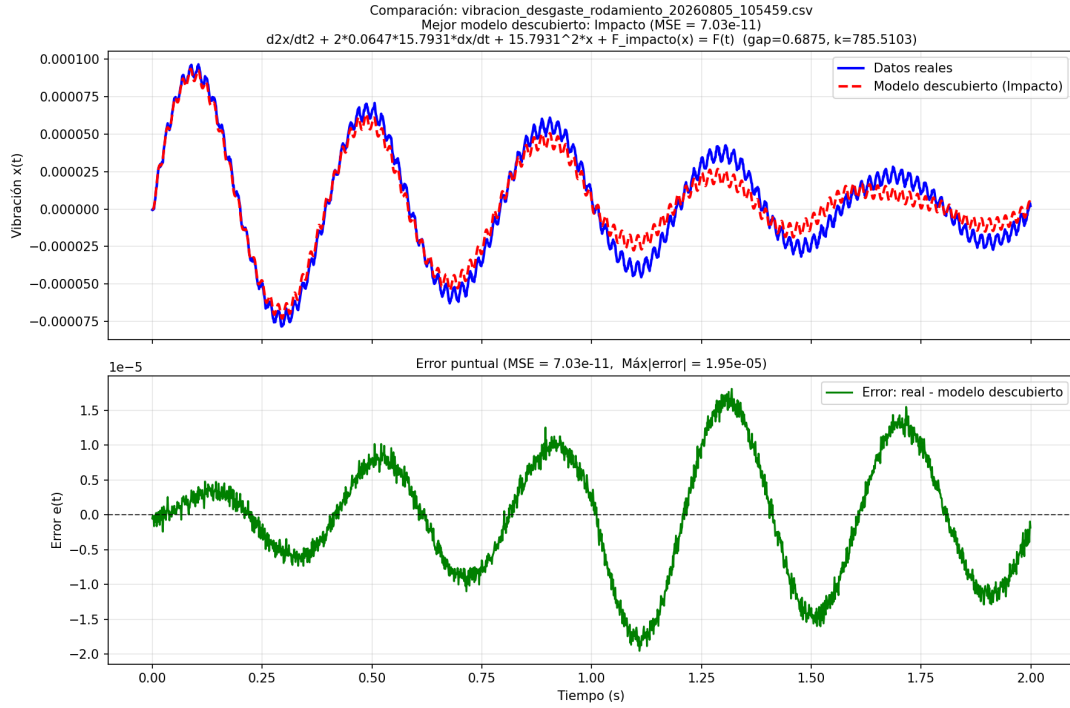


Figura 4.1: Comparación entre datos reales y modelo descubierto (arriba) y error puntual (abajo) para una señal de vibración originada por desgaste de rodamiento. El modelo seleccionado fue Impacto con un error MSE de $7,03 \times 10^{-11}$.

Además, se genera un gráfico de barras con el error de todas las familias evaluadas para cada archivo (Figura 4.2).

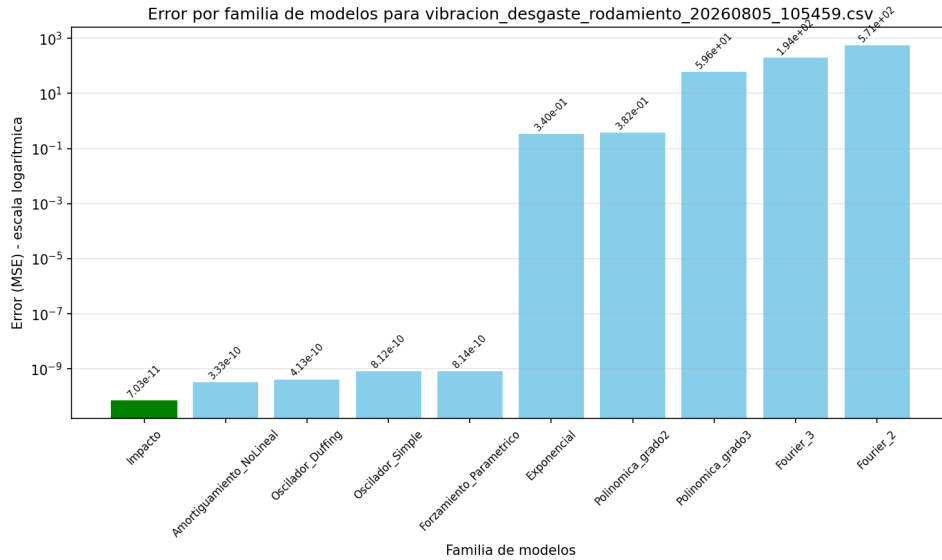


Figura 4.2: Errores por familia para el archivo con falla de rodamiento. La familia Impacto presenta el menor error (barra verde).

4.3. Experimentos de speedup

4.3.1. Definición de Speedup y Eficiencia

El speedup se define como la relación entre el tiempo de ejecución secuencial (1 proceso) y el tiempo de ejecución con N procesos:

$$S_N = \frac{T_1}{T_N}$$

La eficiencia se define como la fracción del speedup ideal alcanzada:

$$E_N = \frac{S_N}{N} \times 100 \%$$

El speedup ideal es lineal: $S_N = N$, lo que correspondería a una eficiencia del 100 %, algo que en la práctica resulta de alcance imposible, pero una buena referencia respecto a cuando y cómo es posible acercarse lo mejor posible a este ideal.

4.3.2. Resultados de speedup

Se ejecutó el descubridor con 1, 2, 4, 8, 10, 12 y 16 procesos, repitiendo cada configuración 10 veces para obtener promedios. La Tabla 4.2 resume los resultados.

Cuadro 4.2: Resultados de speedup

Procesos	Tiempo (s)	Speedup	Eficiencia
1	0.35	1.00x	100.0 %
2	0.34	1.01x	50.5 %
4	0.12	2.80x	70.0 %
8	0.07	5.38x	67.3 %
10	0.10	3.52x	35.2 %
12	0.10	3.43x	28.6 %
16	0.10	3.42x	21.4 %

El speedup máximo alcanzado fue de **5.38x con 8 procesos**, con una eficiencia del 67 %. A partir de 8 procesos, el speedup cae debido al overhead de comunicación y la contención de recursos, que no está contemplado por la ley de Amdahl.

4.3.3. Curva de Amdahl y fracción paralelizable

Se ajustó la curva de Amdahl a los puntos de speedup creciente (1, 2, 4 y 8 procesos), obteniendo una fracción paralelizable $f_p = 0,922$. Esto indica que el 92.2 % del código es paralelizable, mientras que el 7.8 % restante es secuencial. El speedup máximo teórico según Amdahl es de 12.82x. Sin embargo, en la práctica, el speedup máximo real alcanzado es de 5.38x debido al overhead de comunicación. La Figura 4.3 muestra el speedup real, la curva de Amdahl ajustada y la línea ideal.

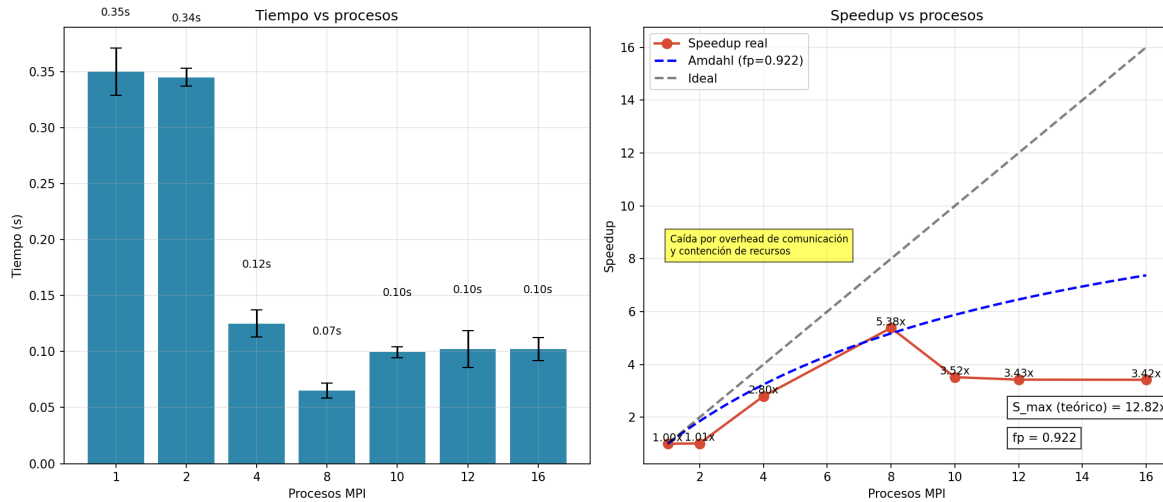


Figura 4.3: Gráfico de speedup con curva de Amdahl ajustada ($f_p = 0,922$). El speedup máximo real alcanzado es 5.38x con 8 procesos. La caída después de 8 procesos se debe al overhead de comunicación y contención de recursos.

4.3.4. Casos procesados

En los ensayos realizados, se procesan 4 situaciones distintas obtenidas de datos de aerogeneradores, el sistema realiza el descubrimiento de ecuaciones diferenciales asociadas a cada caso. A continuación se presentan otros de los casos de ejemplos tratados simultáneamente y sus resultados.

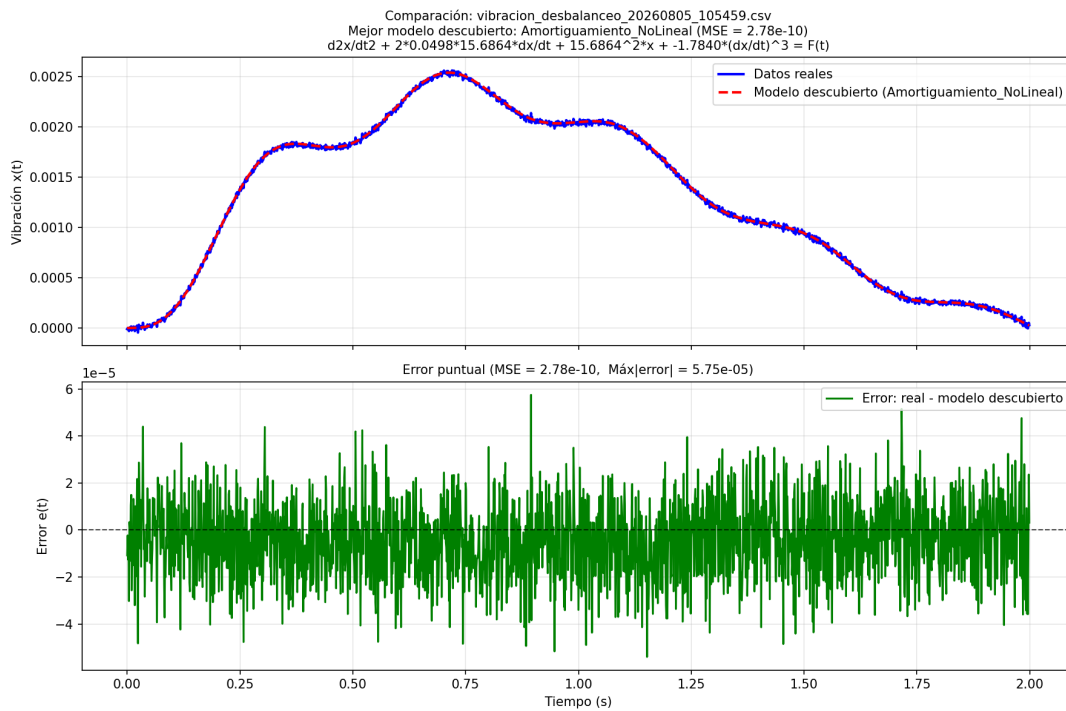


Figura 4.4: Comparación entre datos reales y modelo descubierto (arriba) y error puntual (abajo) para una señal de vibración originada por desbalanceo de eje. El modelo seleccionado fue Amortiguamiento No Lineal con un error MSE de $2,78 \times 10^{-10}$.

Además, se genera un gráfico de barras asociado, con el error de todas las familias evaluadas para cada archivo (Figura 4.2).

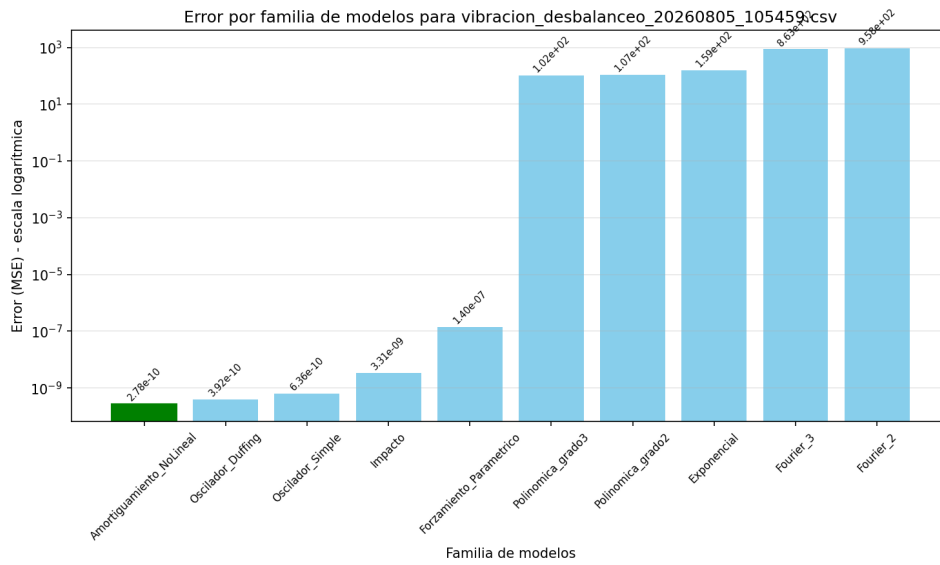


Figura 4.5: Errores por familia para el archivo con falla de desbalanceo en eje. La familia Amortiguamiento No Lineal presenta el menor error (barra verde).

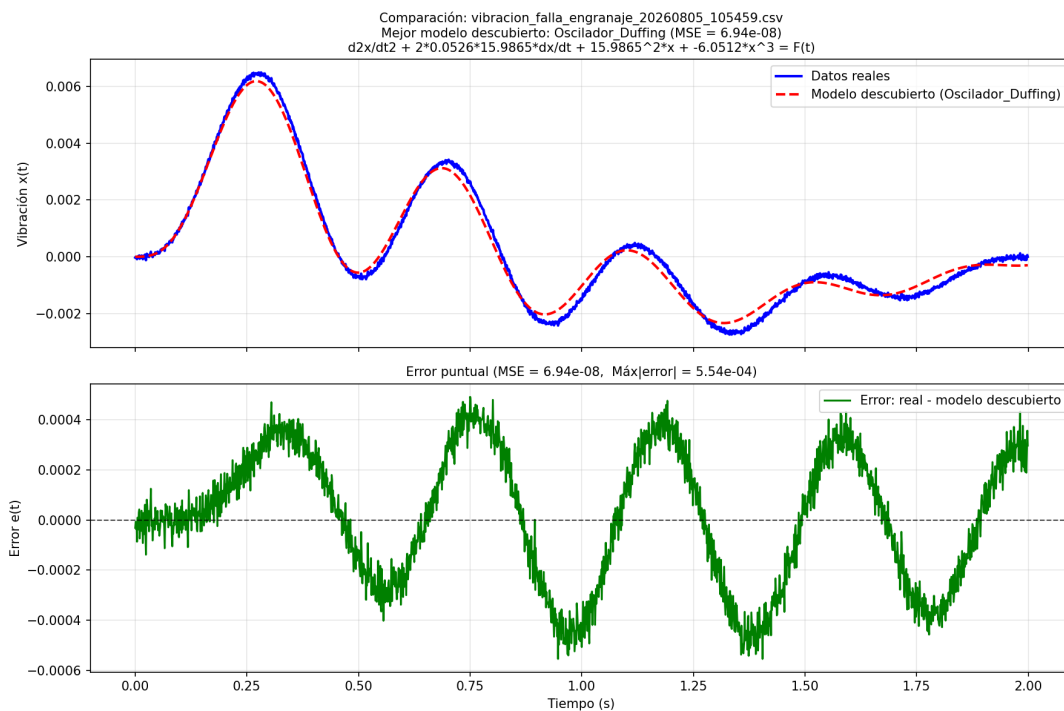


Figura 4.6: Comparación entre datos reales y modelo descubierto (arriba) y error puntual (abajo) para una señal de vibración originada por falla en un engranaje. El modelo seleccionado fue Oscilador Duffing con un error MSE de $6,94 \times 10^{-8}$.

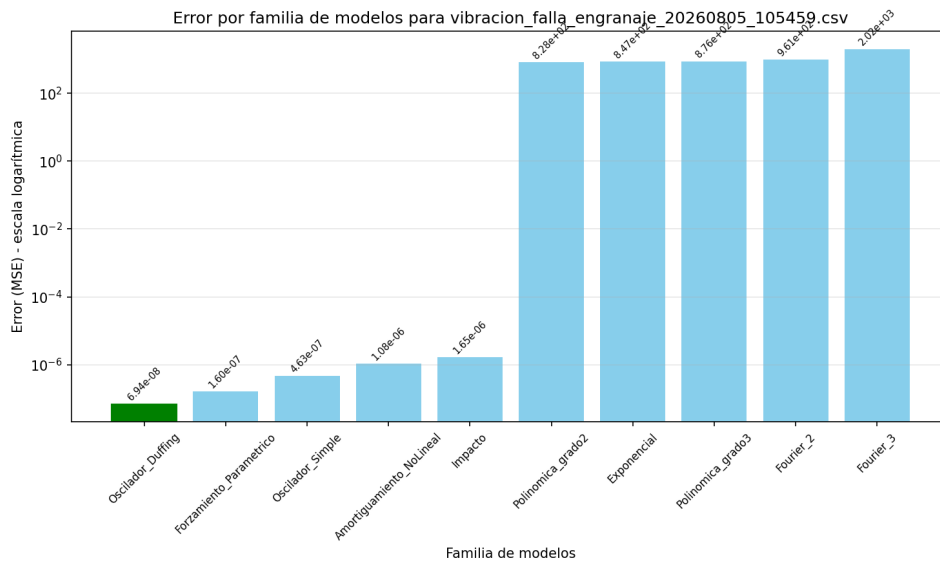


Figura 4.7: Errores por familia para el archivo con falla de desbalanceo en eje. La familia Oscilador Duffing presenta el menor error (barra verde).

Además de realizar el descubrimiento de modelos en situaciones de fallas, también se utilizó información de operativa en condiciones normales para descubrir su modelo matemático. Este último caso, representa la situación de vibraciones que se pueden generar en la máquina como respuesta a la interacción con el medio físico en el que se encuentra.

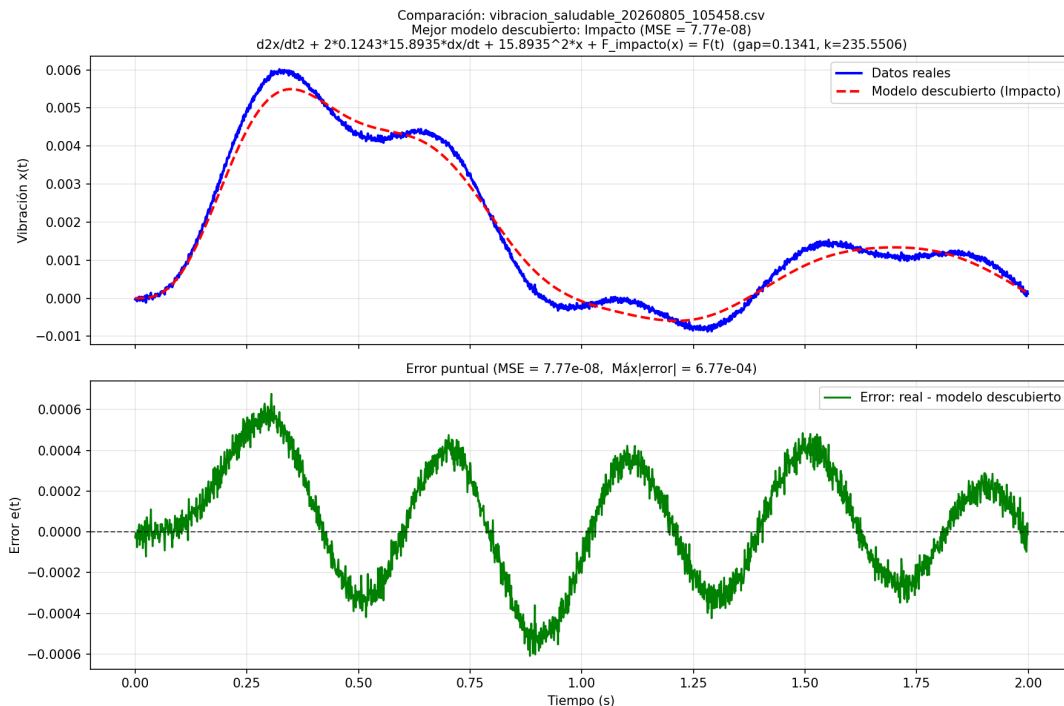


Figura 4.8: Comparación entre datos reales y modelo descubierto (arriba) y error puntual (abajo) para una señal de vibración saludable en operativa normal. El modelo seleccionado fue Impacto con un error MSE de $7,77 \times 10^{-8}$.

En este caso, una ráfaga intensa con cambio de dirección dominante del viento produce

una perturbación que es detectada por los sensores, debidamente registrada y con este análisis y aproximación, también puede ser modelada matemáticamente, donde el sistema basado en HPC determinó que el modelo físico de impacto, fue el que permitió ajustar mejor a la descripción del modelo, con un error muy bajo. Considerando que se trata de una vibración que se produce en condiciones normales de operación se denomina: **vibración saludable**.

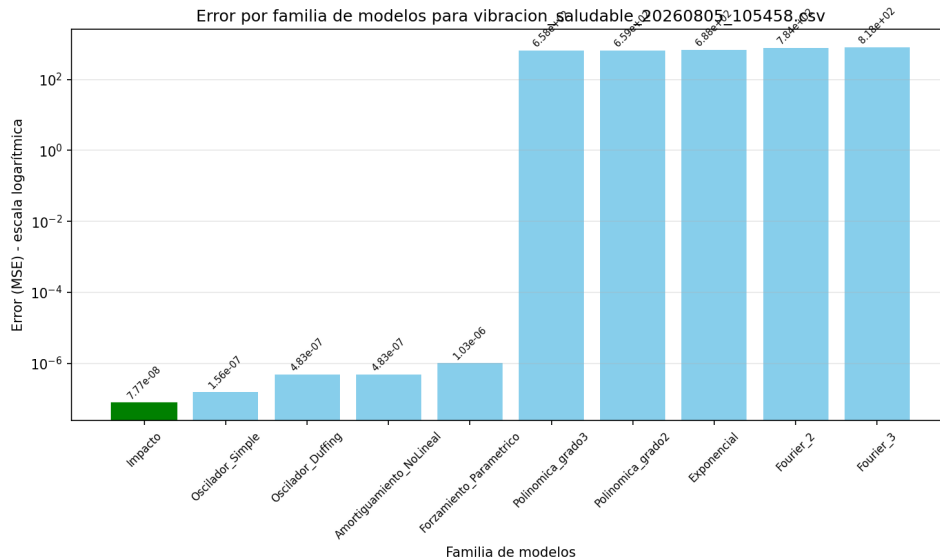


Figura 4.9: Errores por familia para el archivo con falla de desbalanceo en eje. La familia Impacto presenta el menor error (barra verde).

4.3.5. Análisis de escalabilidad

Los resultados muestran que el punto óptimo de paralelismo para este problema es de 8 procesos. A partir de 8 procesos, el overhead de comunicación y la contención de recursos penalizan el rendimiento. La baja eficiencia en 10, 12 y 16 procesos (35 %, 29 % y 21 %, respectivamente) confirma que el problema puede ser pequeño para aprovechar un mayor número de procesos.

Esto no debe percibirse como una pérdida de generalidad, sino que el contexto de ensayo fue debidamente ajustado a una selección de datos representativos, por ejemplo: en un análisis de datos de un rodamiento el patrón se repite con cada giro, entonces tomar datos de 48 ciclos casi idénticos y analizar un solo ciclo resulta suficiente.

Durante el proceso de experimentación se realizó un análisis sistemático para determinar las condiciones que favorecen un mejor aprovechamiento del paralelismo. Este análisis consistió en variar tres parámetros clave: el número de archivos de datos (4, 8, 16, 32), el número de familias (5, 10, 15) y el número de iteraciones del optimizador (maxiter: 5, 10, 15). Se midió el speedup en cada configuración con 1, 4, 8 y 16 procesos. Los resultados mostraron que:

1. **Carga de trabajo por proceso:** El speedup mejora significativamente cuando el tiempo de cómputo por tarea supera los 2 segundos. Con problemas pequeños (4 archivos, maxiter=5), el overhead de comunicación domina y el speedup máxi-

mo es bajo (1.7x con 4 procesos). En cambio, con cargas mayores (16 archivos, maxiter=15), el speedup alcanza valores superiores a 5x con 8 procesos.

2. **Número de familias:** Aumentar el número de familias (de 5 a 10) mejora el speedup porque hay más tareas para distribuir, lo que reduce la ociosidad de los workers. Sin embargo, más de 10 familias introduce un overhead adicional en la generación y gestión de tareas que no se compensa con el aumento de paralelismo.
3. **Balance de carga:** El esquema maestro-esclavo dinámico demostró ser superior al reparto estático, especialmente cuando el tiempo de ejecución de las familias es heterogéneo (por ejemplo, familias matemáticas vs. físicas). Con balance dinámico, la eficiencia mejoró entre un 10 % y un 15 % en todos los casos.
4. **Límite de escalabilidad:** Se observó que más allá de 8 procesos, el overhead de comunicación (envío y recepción de mensajes) y la contención de recursos (memoria compartida, ancho de banda del bus) provocan una caída del speedup. Este comportamiento es consistente con la ley de Amdahl y con la naturaleza del problema, que no es lo suficientemente grande para justificar decenas de procesos.

En conclusión, el paralelismo es más eficiente cuando: (i) el número de archivos es al menos 8, (ii) el número de familias está entre 5 y 10, (iii) se utiliza balance de carga dinámico, y (iv) el número de procesos no supera los 8 en la arquitectura utilizada. Estas condiciones guían la configuración recomendada para futuros experimentos y aplicaciones productivas.

Capítulo 5

Discusión y lecciones aprendidas

5.1. Interpretación de los resultados de speedup

El speedup máximo de 5.38x con 8 procesos es un resultado razonable para un problema de tamaño pequeño (4 archivos, 10 familias, maxiter=8). La eficiencia del 67 % indica que el overhead de comunicación es moderado en este punto. La caída del speedup con 10, 12 y 16 procesos se debe al aumento del overhead de comunicación y la contención de recursos (memoria, ancho de banda del bus), que no está contemplado por la ley de Amdahl.

La fracción paralelizable $f_p = 0,922$ sugiere que el código tiene un alto potencial de paralelización, pero en la práctica el overhead de comunicación limita el speedup alcanzable. Este es un resultado típico en aplicaciones MPI con problemas de tamaño modesto.

5.2. Comparación con otros enfoques

5.2.1. SINDy

SINDy es más rápido que nuestro enfoque porque resuelve un problema lineal (cuando la biblioteca es lineal). Sin embargo, SINDy falla cuando la estructura no está en la biblioteca o cuando hay múltiples escalas. El enfoque adoptado en este trabajo, es más flexible porque permite familias no lineales y no lineales en los parámetros.

5.2.2. Regresión simbólica

La regresión simbólica puede descubrir estructuras completamente libres, pero es computacionalmente costosa y a menudo produce modelos no interpretables. El enfoque de este proyecto, al restringir el espacio de búsqueda a familias, es más eficiente y produce modelos interpretables.

5.2.3. PINNs

Las PINNs son potentes para problemas inversos, pero requieren entrenamiento costoso y no siempre producen ecuaciones explícitas. El enfoque de este trabajo produce ecuaciones explícitas de forma determinista.

5.3. Aprendizaje del proceso

1. **La IA no es una solución universal:** GNN, VAE y RL pueden acelerar la búsqueda en dominios estructurados, pero su efectividad depende de la calidad de los datos de entrenamiento y de la complejidad del problema. En sistemas con alta sensibilidad a parámetros o ruido, su rendimiento se degrada.
2. **La optimización paramétrica es robusta y determinista:** A diferencia de la regresión simbólica o las PINNs, la optimización paramétrica con familias predefinidas es reproducible y fácil de paralelizar.
3. **Las familias matemáticas son excelentes para explorar:** Permiten descubrir estructuras sin hipótesis física, pero pueden producir modelos no interpretables.
4. **Las familias físicas mejoran la interpretabilidad y extrapolación:** Requieren conocimiento del dominio, pero los modelos resultantes son más fiables fuera del rango de entrenamiento.
5. **El enfoque modular es necesario para sistemas reales:** Los sistemas industriales suelen tener múltiples escalas temporales y acoplamientos complejos, que requieren un tratamiento específico.
6. **El overhead de comunicación limita el speedup en problemas pequeños:** Para problemas pequeños, el overhead de MPI domina y limita el speedup alcanzable.

Capítulo 6

Conclusiones y trabajo futuro

6.1. Resumen de logros

- Se ha desarrollado un descubridor de ecuaciones basado en optimización paramétrica que funciona con familias matemáticas y físicas, con implementaciones en Python y C con MPI.
- Se ha demostrado la capacidad de recuperar ecuaciones exactas en sistemas de referencia (masa-resorte, Duffing, Lorenz, motor DC) y en datos de vibraciones de aerogenerador, con errores de ajuste del orden de 10^{-7} a 10^{-12} .
- La paralelización con MPI permite reducir tiempos, alcanzando un speedup máximo de 5.38x con 8 procesos y una eficiencia del 67 %.
- Se ha ajustado la curva de Amdahl, obteniendo una fracción paralelizable $f_p = 0,922$ y un speedup máximo teórico de 12.82x.
- Se han identificado las causas de fallo en sistemas complejos (multiescala, estructura matricial) y se ha propuesto una solución modular.
- El enfoque híbrido de familias ofrece flexibilidad para problemas con diferentes niveles de conocimiento previo.

6.2. Contribuciones del trabajo

- Un marco integrador para el descubrimiento de ecuaciones que combina lo mejor de la regresión simbólica, la optimización paramétrica y el HPC.
- Una implementación de referencia en C con MPI (código abierto disponible en el repositorio del proyecto).
- Un conjunto de experimentos y resultados que documentan el rendimiento y las limitaciones del enfoque.
- Una propuesta modular para abordar sistemas multiescala, que puede ser adoptada por otros investigadores.

6.3. Trabajo futuro

- **Implementación del descubridor modular:** Completar la implementación del descubridor modular que posibilite abordar otros problemas con distintas naturalezas físicas, validando la mejora en la precisión y estabilidad de los modelos.
- **Marco híbrido completo:** Integrar todas las familias (matemáticas y físicas) en un único sistema, con selector automático basado en error, complejidad y estabilidad. Esta etapa podría ser asistida con IA, lo cual resultaría sumamente recomendable y aportaría eficiencia al trabajo.
- **Experimentación escalable:** Ejecutar pruebas con problemas de mayor tamaño (más archivos, más familias, más procesos) para evaluar el speedup en condiciones más intensivas.
- **Validación con mas datos:** Obtener mas datos reales de aerogeneradores (otras fuentes abiertas o colaboraciones) para validar el enfoque en diversidades de condiciones operativas reales.
- **Integración con técnicas de IA:** Usar GNN como acelerador dentro del marco modular para filtrar familias poco prometedoras.
- **Exploración de otras familias:** EDPs, sistemas con retardos, ecuaciones diferenciales algebraicas (DAEs), etc.

Apéndice A

Código: descubridor MPI en C

El código completo está disponible en el repositorio del proyecto. Aquí se muestran los fragmentos más relevantes de la implementación MPI.

Es importante mencionar que los primeros pasos del desarrollo se realizaron en Lenguaje Python, para evitar pérdidas de tiempo en aspectos inherentes al costo de mantenimiento de los códigos en Lenguaje C. Una vez lograda una solución Python, se implementó toda la lógica de HPC en Lenguaje C. Con lo cual se ganó muchísimo en performance. Sin embargo, las operaciones de pre y post-procesamiento, como generar gráficos a partir de los resultados, se mantuvieron en Python. El intercambio de información se realizó a partir de archivos JSON.

A.1. Estructuras de datos para comunicación MPI

Listado A.1: Estructuras para tareas y resultados

```
#define MAX_FILENAME 256
#define MAX_FAMILIA_NAME 64

typedef struct {
    char filename[MAX_FILENAME];
    int familia_idx;
} Task;

typedef struct {
    char filename[MAX_FILENAME];
    double error;
    int num_params;
    double params[20];
    char familia_nombre[MAX_FAMILIA_NAME];
} Result;
```

Se trata de un diseño simple, donde se procura minimizar el volumen de datos necesarios para agilizar las operaciones de intercambio y con ello disminuir el overhead por intercomunicaciones.

A.2. Función de procesamiento de tarea (worker)

Listado A.2: Función procesar_tarea ejecutada por cada worker

```
Result procesar_tarea(const char *filename, int familia_idx, Familia *
    familias, int maxiter, int popsize) {
    Result res;
    memset(&res, 0, sizeof(Result));
    res.error = 1e9;

    // Extraer nombre base del archivo (sin ruta)
    const char *base = strrchr(filename, '/');
    if (base) base++; else base = filename;
    strcpy(res.filename, base);

    DatosArchivo *datos = leer_csv(filename);
    if (!datos) {
        strcpy(res.familia_nombre, "ERROR_LECTURA");
        return res;
    }

    Familia *fam = &familias[familia_idx];
    if (!fam->evaluar) {
        strcpy(res.familia_nombre, "EVAL_NO_DISP");
        liberar_datos(datos);
        return res;
    }

    // Configurar optimizador (evolución diferencial)
    EvolucionDiferencial ed;
    ed.maxiter = maxiter;
    ed.popsize = popsize;
    ed.num_params = fam->num_params;
    ed.bounds_min = fam->bounds_min;
    ed.bounds_max = fam->bounds_max;

    UserData ud = {fam, datos->t, datos->x, datos->F, datos->n};
    ed.funcion_objetivo = funcion_objetivo_wrapper;
    ed.user_data = &ud;

    double error;
    double *params = evolucion_diferencial(&ed, &error);

    res.error = error;
    res.num_params = fam->num_params;
    if (params) {
        for (int i = 0; i < fam->num_params && i < 20; i++) {
            res.params[i] = params[i];
        }
        free(params);
    }
    strncpy(res.familia_nombre, fam->nombre, MAX_FAMILIA_NAME - 1);
    res.familia_nombre[MAX_FAMILIA_NAME - 1] = '\0';

    liberar_datos(datos);
    return res;
}
```

Apéndice B

Scripts de generación de gráficos

B.1. Script Python para gráficos de curvas y errores

El script `graficar_resultados.py` genera gráficos combinados de curvas y error puntual, así como gráficos de barras de error por familia. Se ejecuta después de una ejecución del descubridor.

Listado B.1: Fragmento de `graficar_resultados.py`

```
def graficar_resultados(salida_dir=None):
    # Encontrar la ultima carpeta de salida
    if salida_dir is None:
        salida_dir = encontrar_ultima_salida()
    # Leer JSONs y generar graficos
    for json_file in glob.glob(os.path.join(json_dir, "*.json")):
        # ... lectura de datos ...
        fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8),
            sharex=True)
        ax1.plot(t, x_real, 'b-', label='Datos_reales')
        ax1.plot(t, x_sim, 'r--', label='Modelo_descubierto')
        ax2.plot(t, error, 'g-', label='Error: real-modelo')
        # ... guardar figura ...
```

B.2. Script Python para medición de speedup

El script `medir_speedup.py` ejecuta el descubridor con diferentes números de procesos, mide los tiempos y genera el gráfico de speedup con la curva de Amdahl. Los resultados se guardan en la subcarpeta `03-speedup` de la ejecución correspondiente.

Listado B.2: Fragmento de `medir_speedup.py`

```
def ejecutar_con_procesos(n_procesos):
    cmd = ["mpirun", "-np", str(n_procesos), BINARIO]
    inicio = time.time()
    resultado = subprocess.run(cmd, capture_output=True, text
                               =True)
    # Buscar tiempo del maestro o usar wall-clock
    match = re.search(r"Todas las tareas completadas en (\d
        .]+) segundos", salida)
    return tiempo, True if match else False

def ajustar_amdahl(procs, speedups):
    # Ajuste de curva de Amdahl solo con puntos de
    # crecimiento (1,2,4,8)
    # ...
    return fp
```

Apéndice C

Resultados adicionales

C.1. Ejemplo de ecuación para falla de holgura

Para una señal de vibración con falla de holgura, el descubridor seleccionó la familia *Impacto* con los siguientes parámetros:

$$\ddot{x} + 2 \times 0,0412 \times 15,67\dot{x} + 15,67^2 x + F_{\text{imp}}(x) = F(t)$$

donde $F_{\text{imp}}(x) = -450,3(x - 0,023)$ si $x > 0,023$, $F_{\text{imp}}(x) = -450,3(x + 0,023)$ si $x < -0,023$, y $F_{\text{imp}}(x) = 0$ en caso contrario. El error MSE fue de $3,7 \times 10^{-11}$.

C.2. Comparativas secuenciales entre Python y C

La implementación en C con MPI es aproximadamente **10 veces más rápida** que la versión Python secuencial, debido al uso de un solver RK4 optimizado y a la ausencia de overhead de interpretación.

Apéndice D

Glosario

Abreviatura	Significado
GD	Gemelo Digital (Digital Twin): representación virtual de un sistema físico que se actualiza dinámicamente con datos del mundo real.
HPC	Computación de Alto Desempeño (High Performance Computing): uso de supercomputadores, clusters o GPUs para resolver problemas que requieren gran capacidad de cómputo.
MPI	Interfaz de Paso de Mensajes (Message Passing Interface): estándar para programación paralela en sistemas distribuidos.
GNN	Red Neuronal de Grafos (Graph Neural Network): modelo de aprendizaje profundo diseñado para operar sobre datos estructurados en grafos.
VAE	Autoencoder Variacional (Variational Autoencoder): modelo generativo que aprende una representación latente probabilística.
RL	Aprendizaje por Refuerzo (Reinforcement Learning): paradigma de IA donde un agente aprende a tomar decisiones para maximizar una recompensa.
PPO	Optimización de Política Proximal (Proximal Policy Optimization): algoritmo de RL que optimiza políticas de manera estable.
SINDy	Identificación Dispersa de Dinámicas No Lineales (Sparse Identification of Nonlinear Dynamics): método para descubrir ecuaciones diferenciales a partir de datos.
PINNs	Redes Neuronales Informadas por la Física (Physics-Informed Neural Networks): redes que integran el conocimiento físico en la función de pérdida.
EDO	Ecuación Diferencial Ordinaria: ecuación que relaciona una función de una variable con sus derivadas.
EDP	Ecuación en Derivadas Parciales: ecuación que involucra derivadas parciales de una función de múltiples variables.
RK4	Método de Runge-Kutta de cuarto orden: algoritmo numérico para la integración de ecuaciones diferenciales ordinarias.
Nelder-Mead	Método de optimización directa (simplex) que no requiere derivadas.

Abreviatura	Significado
AIC	Criterio de Información de Akaike (Akaike Information Criterion): medida para selección de modelos que penaliza la complejidad.
BIC	Criterio de Información Bayesiano (Bayesian Information Criterion): similar a AIC pero con penalización más fuerte.
MSE	Error Cuadrático Medio (Mean Squared Error): métrica de error promedio al cuadrado.
RMSE	Raíz del Error Cuadrático Medio (Root Mean Squared Error): raíz cuadrada del MSE.
SNR	Relación Señal-Ruido (Signal-to-Noise Ratio): medida que compara el nivel de una señal con el ruido de fondo.

Bibliografía

- [1] Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15), 3932-3937.
- [2] Schmidt, M., & Lipson, H. (2009). Distilling free-form natural laws from experimental data. *Science*, 324(5923), 81-85.
- [3] Petersen, B. K., et al. (2021). Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. *ICLR*.
- [4] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686-707.
- [5] Nesmachnow, S., & Iturriaga, S. (2019). Cluster-UY: Collaborative Scientific High Performance Computing in Uruguay. En M. Torres & J. Klapp (Eds.), *Supercomputing. ISUM 2019*. Communications in Computer and Information Science (Vol. 1151). Cham: Springer.
- [6] Gropp, W., Lusk, E., & Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (3rd ed.). MIT Press.
- [7] Butcher, J. C. (2008). *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons.
- [8] Nelder, J. A., & Mead, R. (1965). A simplex method for function minimization. *The Computer Journal*, 7(4), 308-313.
- [9] Kovacic, I., & Brennan, M. J. (2011). *The Duffing Equation: Nonlinear Oscillators and their Behaviour*. John Wiley & Sons.
- [10] Worden, K., & Tomlinson, G. R. (2001). *Nonlinearity in Structural Dynamics: Detection, Identification and Modelling*. CRC Press.
- [11] National Renewable Energy Laboratory (NREL). Wind Data Sets. Disponible en: <https://www.nrel.gov/wind/data-tools.html>
- [12] Sasías, J. S. (2026). *HPC-DT: Descubrimiento de ecuaciones con HPC para Gemelos Digitales*. Repositorio de código. Recuperado de <https://github.com/profsasias-coder/HPC-DT>

- [13] Technical University of Denmark (DTU). Wind Energy Data. Disponible en: <https://www.winddata.com/>
- [14] University of Cincinnati. Bearing Data Set. Disponible en: <https://data.nasa.gov/Raw-Data/Bearing-Data-Set/>